

Planning and Design of the QueueMaster Car Wash Management System

Introduction

Car wash businesses in urban centers often struggle with operational inefficiencies that directly affect customer satisfaction and profitability. In Witbank (Emalahleni), South Africa, many car washes rely on manual queue management systems that use paper tickets, verbal communication, or staff memory to track vehicles. These outdated methods frequently result in long waiting times, duplicated services, miscommunication among staff, and ultimately, dissatisfied customers. To address these challenges, the QueueMaster Car Wash Management System was conceived as a comprehensive digital solution. The system was designed to streamline queue management, enable live monitoring of service progress, and reduce client complaints through transparency and efficiency.

Problem Identification

During the initial research phase, the development team conducted field studies at several car washes in Witbank Central. The observations revealed recurring issues:

Customers lacked visibility into their position in the queue.

Staff struggled to track vehicles across service stages such as pre-wash, washing, drying, detailing, and collection.

Managers had no real-time oversight of operations or accurate daily financial reports.

Miscommunication often led to duplicated services, arguments, and negative reviews.

These inefficiencies not only frustrated customers but also reduced throughput and revenue. The need for a digital system that could provide transparency, improve coordination, and generate reliable reports became evident.

Requirements Gathering

Based on the problems identified, the team outlined both functional and non-functional requirements.

Functional Requirements

Real-time queue tracking from vehicle entry to exit.

Role-based access for different users: Admin, Washer/Staff, and Public Display.

Online booking capability to manage workload in advance.

Live dashboard for staff to update service status instantly.

Financial reporting and archiving of sales transactions.

Secure authentication and authorization using modern security protocols.

Professional user interface optimized for desktop, tablet, and large TV displays.

Non-Functional Requirements

Performance: Low latency for live updates.

Usability: Interface readable under bright outdoor conditions.

Reliability: Stable system with minimal downtime.

Ease of deployment: Simple setup using allowed technologies (Spring Boot, React.js, H2 database).

System Architecture Design

QueueMaster was designed using a modern client-server architecture with a clear separation of concerns. The system follows a RESTful API design pattern, enabling loose coupling between frontend and backend components. This allowed parallel development: while some team members worked on the React.js frontend, others focused on the Spring Boot backend.

Backend Design (Spring Boot + H2 Database)

The backend was implemented using Spring Boot, chosen for its robustness, dependency injection, and support for REST APIs. Data persistence was handled by Spring Data JPA with Hibernate, interacting with the H2 in-memory database (configured to persist to file during development).

Core entities included:

User/Staff – with roles (ADMIN, WASHER).

Vehicle/Booking – containing registration number, model, service type, status, and timestamps.

QueueEntry – tracking current stage and assigned staff.

Sale/Transaction – recording financial data.

The backend followed a layered architecture:

Controller Layer – REST endpoints (/api/queues, /api/bookings, /api/staff, /api/reports).

Service Layer – Business logic and transaction management.

Repository Layer – JPA repositories for database operations.

Security Layer – JWT-based authentication with role-based access control (RBAC).

For real-time updates, the team implemented Server-Sent Events (SSE) to push live queue changes to connected clients, especially for public TV displays and staff dashboards.

Frontend Design (React.js)

The frontend was built as a Single Page Application (SPA) using React.js with react-router-dom for navigation. Key pages included:

Home – Public landing page with service information.

Live Queue/Tracker -optimized view showing queue status.

Admin/Staff Dashboard – Role-protected area for updating vehicle status.

Manage Staff – Admin-only staff management.

Reports – Financial overview and statistics.

Login/Signup – Secure authentication.

The UI/UX design followed a Slate Blue/Dark theme with high contrast colors (--primary: #0f172a, --accent: #3b82f6). Large readable fonts, color-coded status pills (waiting, washing, ready), and responsive grid layouts ensured usability in busy wash bay environments. Animations such as pulse and blink effects highlighted vehicles in active service stages.

Detailed Feature Design

Live Queue Management

Vehicles move through stages: Waiting → In Progress → Ready. Staff can advance a vehicle's status with one click, instantly updating all connected clients. Initially, updates were handled, with SSE planned for smoother real-time synchronization.

Role-Based Security

Strict access control was implemented. React components such as `ProtectedRoute` and `AdminRoute`, combined with backend JWT validation, ensured that only authorized personnel could update queues or view reports.

Reporting and Analytics

The Reports page displayed KPIs such as: Vehicles served per day. Average service time. Daily revenue.

This enabled managers to identify bottlenecks and optimize operations.

Technology Stack Decisions

The project adhered to strict rules: no NetBeans, no JEE frameworks, no Glassfish. Development was conducted using IntelliJ IDEA and VS Code.

Backend: Spring Boot with Maven for dependency management.

Database: H2 Console for rapid development and demonstration.

Frontend: React.js with Axios for API communication.

Collaboration: GitHub repository with feature branches, README documentation, and Postman API collections.

Design Challenges and Solutions

Real-time synchronization: WebSockets were considered but deemed too complex for the project timeline. A hybrid approach of REST + polling + SSE was adopted.

Visibility under sunlight: High-contrast colors and large typography were chosen to ensure readability outdoors.

Usability testing: Simulated users helped refine workflows, reducing clicks needed to update vehicle status and simplifying the booking form.

Benefits and Expected Impact

QueueMaster directly addresses client complaints by:

Providing transparency: Customers can see their position in the queue.

Reducing waiting uncertainty: Live updates reassure clients.

Improving service speed: Staff coordination is streamlined.

Enabling data-driven decisions: Reports guide management strategies.

The system is expected to increase daily throughput, improve customer satisfaction, and professionalize car wash operations in Witbank.

Conclusion and Reflection

The planning and design of QueueMaster demonstrate how thoughtful Technology can transform local business operations. By starting with real-world problems, the team created a solution that is technically sound and practically valuable. The modular architecture, robust security, professional UI/UX, and focus on live queue visibility reflect industry-standard practices.

This essay explores the planning, design, and implementation of QueueMaster, a web-based platform developed using Spring Boot (Java framework) for the backend, H2 Console database for data storage, and React.js (JavaScript) for the frontend. The discussion highlights the problem identification, requirements gathering, architectural design, feature implementation, and expected impact of the system.

This project strengthened the team's understanding of full-stack development, requirements engineering, architectural design, and real-time application design. QueueMaster stands as proof that deliberate planning and design can turn chaotic manual processes into efficient, customer-friendly digital systems.